19 BERT

트랜스포머의 인코더만 쌓아, 방대한 글에서 "빈칸 맞히기"로 언어를 미리 배워 둔 모델입니다. 이미 학습된 BERT를 내려받아 빈칸을 채우게 하고, 두 문장이 이어지는지 묻고, 문장의 뜻을 벡터로 바꿔 봅니다.

오늘 다루는 것

1. 자연어 처리에서의 사전 훈련
2. 버트(BERT): 구조, 입력, 사전 훈련, 파인 튜닝
3. BERT의 마스크드 언어 모델
4. BERT를 이용한 다음 문장 예측
5. 문장과 문서 임베딩을 위한 SBERT

끝나고 나면 할 수 있는 것

1. 사전 훈련과 파인 튜닝이 무엇이고 왜 두 단계로 나누는지 설명할 수 있다.
2. BERT의 입력이 세 가지 임베딩의 합이라는 것과 [CLS], [SEP], [MASK]의 역할을 말할 수 있다.
3. BERT가 사전 훈련에서 푸는 두 문제(MLM, NSP)를 실제 모델로 실행해 볼 수 있다.

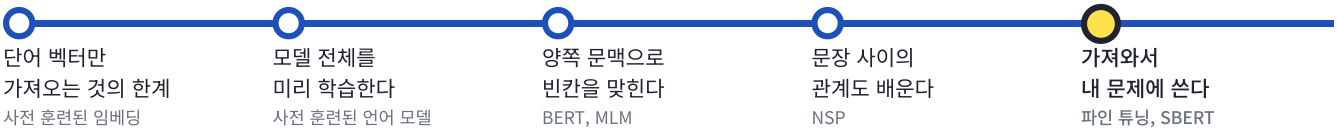
2026년 10월 16일 금요일 14:30 (10/16 자료 1/5)
코랩에는 transformers가 설치되어 있습니다. 실습 5만 `!pip install sentence-transformers` 가 필요합니다. 처음 실행할 때 모델을 내려받습니다(각 440MB 안팎).

19

BERT

13장에서 리뷰 분류기를 만들 때, 모델은 리뷰 수만 개만 보고 단어의 뜻부터 문장을 읽는 법까지 모두 배워야 했습니다. BERT는 순서를 바꿉니다. 먼저 엄청난 양의 글로 언어 자체를 배워 두고(사전 훈련), 그 모델을 가져와 내 문제에 맞게 조금만 더 학습시킵니다(파인 튜닝).

왜 배우나: 오늘의 학습 경로



이 장에서는 BERT가 무엇이고 이미 학습된 BERT가 무엇을 할 수 있는지를 봅니다. BERT를 내 데이터로 더 학습시키는 일(파인 튜닝)은 20장에서 합니다. 예시 문장은 교재의 "Soccer is a really fun [MASK]."와 "축구는 정말 재미있는 [MASK]다."를 따라갑니다.

19-01 자연어 처리에서의 사전 훈련

핵심 한 줄

사전 훈련은 레이블이 없는 방대한 글로 모델을 미리 학습시켜 두는 것이고, 그 대상이 단어 벡터(Word2Vec)에서 문맥을 읽는 모델 전체(ELMo, GPT, BERT)로 넓어져 왔다.

왜 배우나

BERT는 갑자기 나온 것이 아닙니다. 12장의 사전 훈련된 임베딩, ELMo, 18장의 트랜스포머가 차례로 쌓인 끝에 나왔습니다. 무엇이 부족해서 다음 것이 나왔는지를 따라가면, BERT가 왜 그런 모양인지 알 수 있습니다.

1) 단어 벡터에서 언어 모델로

정의

- 사전 훈련(pre-training): 풀려는 문제의 데이터를 쓰기 전에, 다른 큰 데이터로 모델을 미리 학습시켜 두는 것.
- 파인 튜닝(fine-tuning, 미세 조정): 사전 훈련된 모델을 가져와 내 문제의 데이터로 조금 더 학습시키는 것(12-07, 13-05).

단계	미리 학습해 두는 것	한계
사전 훈련된 워드 임베딩 (Word2Vec, GloVe, 12장)	단어마다 벡터 하나	한 단어 = 한 벡터. 문맥을 보지 못해 다의어를 구분하지 못한다
ELMo (12-08)	순방향 LSTM 언어 모델 + 역방향 LSTM 언어 모델	두 방향을 따로 학습해 이어 붙인 것. RNN이라 느리다
GPT-1	트랜스포머 디코더 12층으로 된 언어 모델	앞에서 뒤로만 본다 (단방향)
BERT	트랜스포머 인코더 12층. 가린 단어 맞히기로 학습	글을 이어 쓰는 데는 맞지 않다

왜 언어 모델일까? 다음 단어 맞히기(10장)는 글만 있으면 문제와 정답을 얼마든지 만들 수 있습니다. 사람이 레이블을 붙일 필요가 없어, 위키백과 전체 같은 방대한 글로 학습할 수 있습니다.

왜 양방향 언어 모델은 그냥 만들 수 없을까? 언어의 문맥은 양쪽에 있습니다(14장에서 양방향 LSTM을 쓴 이유). 그런데 "다음 단어 맞히기"를 양방향으로 하면, 뒤에서 오는 쪽이 맞혀야 할 단어를 이미 봐 버립니다. 답을 보고 답을 맞히는 꼴입니다. ELMo는 순방향과 역방향을 따로 학습해 이 문제를 피했습니다.

- 마스크드 언어 모델(Masked Language Model, MLM): 문장의 단어 일부(15%)를 가리고, 가린 단어를 **앞뒤 문맥을 모두 보고** 맞히게 하는 학습 방법. 영어 시험의 빈칸 채우기와 같습니다. 맞힐 단어 자체를 가렸으므로 양쪽을 다 봐도 답이 새지 않습니다. BERT가 쓰는 방법입니다.

예시 적용

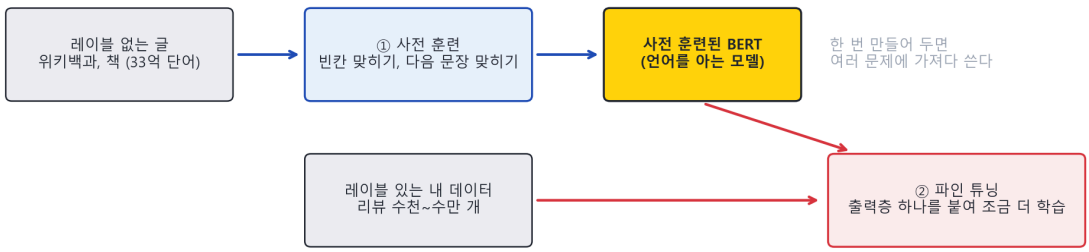


그림 1 사전 훈련과 파인 튜닝. 큰 글로 한 번 학습해 둔 모델을 여러 문제에 가져다 쓴다

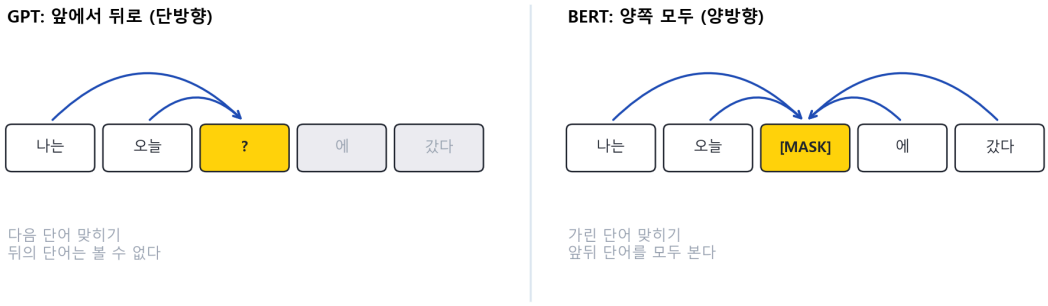


그림 2 다음 단어 맞히기(왼쪽)는 앞의 단어만 본다. 가린 단어 맞히기(오른쪽)는 양쪽을 모두 본다

문맥에 따라 벡터가 달라지는지 확인(실습 1). 사전 훈련된 BERT에 bank가 들어간 문장 셋을 넣고, bank 자리의 벡터끼리 코사인 유사도를 쟀습니다.

bank의 벡터	은행 문장 vs 은행 문장	은행 문장 vs 강둑 문장
BERT에 들어가기 전 (단어 임베딩)	1.000	1.000
BERT를 지난 뒤	0.936	0.300

- 들어가기 전에는 어느 문장에서든 bank의 벡터가 같습니다(1.000). 12장의 임베딩 층이 하는 일이 이것입니다.
- BERT를 지나면 은행끼리는 0.936으로 여전히 비슷하고, 은행과 강둑은 0.300으로 멀어집니다. 주변의 money, river를 보고 뜻을 구분한 것입니다.
- 12-08에서는 학습되지 않은 모델로 구조만 봤습니다. 이번에는 학습된 모델이라 숫자에 뜻이 있습니다.

비유

사전을 외운 사람과 책을 많이 읽은 사람

단어장만 외운 사람은 "배"의 뜻을 하나만 압니다. 책을 많이 읽은 사람은 "배를 타고", "배가 고파서", "배를 깎아"를 보고 그때그때 뜻을 압니다. 새 일을 배울 때도 책을 많이 읽은 사람은 "이 일에서 쓰는 말"만 조금 익히면 됩니다.

단어장 외우기	= = 사전 훈련된 워드 임베딩
책을 많이 읽기	= = 언어 모델의 사전 훈련
문장을 보고 뜻을 아는 것	= = 문맥을 반영한 임베딩
새 일에서 쓰는 말 익히기	= = 파인 튜닝

실습 1 같은 단어 bank의 벡터가 문맥에 따라 달라지는지 확인하기

```
# 코랩에는 transformers가 이미 설치되어 있다. (처음 한 번 모델 440MB 다운로드)
import torch
from transformers import AutoModel, AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")
model.eval()

sentences = ["i deposited money at the bank",          # 은행
             "he borrowed money from the bank",        # 은행
             "we sat on the bank of the river"]        # 강둑

def vectors_of(sentence, word="bank"):
    """(BERT에 들어가기 전의 단어 임베딩, BERT를 지난 뒤의 벡터)"""
    inputs = tokenizer(sentence, return_tensors="pt")
    position = inputs["input_ids"][0].tolist().index(
        tokenizer.convert_tokens_to_ids(word))
    with torch.no_grad():
        before = model.embeddings.word_embeddings(inputs["input_ids"])[0, position]
        after = model(**inputs).last_hidden_state[0, position]
    return before, after

before, after = zip(*[vectors_of(s) for s in sentences])
cos = lambda a, b: torch.cosine_similarity(a, b, dim=0).item()

print("bank의 벡터끼리의 코사인 유사도")
print("                              들어가기 전    BERT를 지난 뒤")
for (i, j), name in [((0, 1), "은행 문장 1 vs 은행 문장 2"),
```

```
                    ((0, 2), "은행 문장 1 vs 강둑 문장  ")]:
    print(f"  {name}     {cos(before[i], before[j]):.3f}        "
          f"{cos(after[i], after[j]):.3f}")
print("벡터의 크기:", tuple(after[0].shape))
```

```
실행 결과
bank의 벡터끼리의 코사인 유사도
                              들어가기 전   BERT를 지난 뒤
  은행 문장 1 vs 은행 문장 2     1.000        0.936
  은행 문장 1 vs 강둑 문장       1.000        0.300
벡터의 크기: (768,)
```

직접 해 보기 문장을 바꿔 보세요. `"the bank raised interest rates"`와 `"fish swim near the bank"`의 bank는 각각 어느 쪽과 더 비슷하게 나오나요?

확인 문제

19-01 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 Word2Vec은 같은 단어라도 문맥에 따라 다른 벡터를 준다. (O / X) 이름 _____

2 언어 모델은 레이블이 없는 글로도 학습할 수 있다. (O / X) 이름 _____

3 마스크드 언어 모델이 맞히는 것은? 이름 _____

① 문장의 다음 단어 ② 가려진 단어 ③ 문장의 감정

4 GPT-1의 구조는? 이름 _____

① 트랜스포머 디코더를 쌓은 것 ② 양방향 LSTM ③ CNN

5 사전 훈련된 모델을 내 데이터로 조금 더 학습시키는 것을 _____이라고 한다. 이름 _____

6 실습에서 은행 문장과 강둑 문장의 bank 벡터는 BERT를 지난 뒤 유사도가 _____이었다. 이름 _____

7 단어 벡터만 미리 학습하는 것과 모델 전체를 미리 학습하는 것의 차이를 단어장과 독서에 빗대어 설명해 보세요. 이름 _____

원문: 19-01 자연어 처리에서의 사전 훈련 — https://wikidocs.net/217012 (교재에는 실습이 없어 문맥에 따른 벡터 차이를 확인하는 실습을 추가했습니다)

19-02 버트(BERT)

핵심 한 줄

BERT는 트랜스포머 인코더를 쌓은 모델로, 단어·위치·세그먼트 임베딩의 합을 입력받아 토큰마다 문맥이 반영된 벡터를 내고, 마스크드 언어 모델과 다음 문장 예측으로 사전 훈련한 뒤 문제에 맞게 파인 튜닝해 쓴다.

왜 배우나

BERT(Bidirectional Encoder Representations from Transformers)는 2018년 구글이 내놓았고, 당시 거의 모든 자연어 처리 문제의 최고 기록을 바꿨습니다. 20장에서 BERT를 직접 학습시키려면 BERT에 무엇을 넣고 무엇이 나오는지를 정확히 알아야 합니다. 입력에 붙는 특별 토큰과 세 가지 입력(input_ids, token_type_ids, attention_mask)이 여기서 나옵니다.

1) 구조와 입력

정의

크기. 18장의 트랜스포머 인코더 층을 그대로 쌓았습니다. 디코더는 없습니다.

	층 수 L	벡터의 크기 D	어텐션 헤드 A	파라미터
트랜스포머 (18장)	6	512	8	
BERT-Base	12	768	12	약 1.1억 개
BERT-Large	24	1024	16	약 3.4억 개

출력. 토큰이 n개 들어가면 벡터도 n개가 나옵니다: $(n,) \rightarrow (n,\ 768)$. 셀프 어텐션을 12번 거쳤으므로, 각 벡터는 문장의 모든 토큰을 참고한 문맥을 반영한 임베딩입니다.

특별 토큰.

- `[CLS]` : 모든 입력의 맨 앞에 붙입니다. 이 자리의 출력 벡터를 "문장 전체의 요약"으로 보고 분류 문제에 씁니다.
- `[SEP]` : 문장의 끝에 붙입니다. 문장 두 개를 넣을 때 경계가 됩니다.
- `[MASK]` : 가린 단어의 자리. `[PAD]` : 패딩. `[UNK]` : 모르는 토큰.

토크나이저. BERT는 15장의 WordPiece 서브워드 토크나이저를 씁니다. 단어 집합은 30,522개입니다. 단어 집합에 없는 단어는 아는 조각으로 쪼개고, 단어 중간에 오는 조각 앞에 ## 을 붙입니다.

임베딩 세 개의 합. BERT의 입력은 토큰마다 세 벡터를 더한 것입니다. 셋 다 학습됩니다.

임베딩	무엇을 나타내나	표의 모양
단어(WordPiece) 임베딩	토큰의 뜻	(30522, 768)
위치(position) 임베딩	몇 번째 토큰인가	(512, 768)
세그먼트(segment) 임베딩	첫째 문장인가 둘째 문장인가	(2, 768)

- 18장의 트랜스포머는 위치를 사인·코사인 식으로 계산했습니다(포지셔널 인코딩). BERT는 위치마다의 벡터를 **학습**합니다. 위치 임베딩이 512개뿐이라 BERT의 입력은 최대 512토큰입니다.
- "문장 두 개"는 꼭 문장 단위가 아닙니다. 질문과 본문처럼 두 덩어리의 글을 뜻합니다. 글이 한 덩어리뿐이면 세그먼트는 모두 0입니다.

토크나이저가 만들어 주는 세 가지 입력.

- `input_ids` : 토큰의 번호. 단어 임베딩을 꺼내는 데 씁니다.
- `token_type_ids` : 첫째 덩어리는 0, 둘째 덩어리는 1. 세그먼트 임베딩을 꺼내는 데 씁니다.
- `attention_mask` : 실제 토큰은 1, 패딩은 0. 어텐션에서 패딩을 보지 않게 합니다(18장의 패딩 마스크).

예시 적용



세 벡터를 더한 것이 BERT의 입력 (토큰 수, 768). [CLS] 문장의 시작이자 분류용, [SEP] 문장의 끝

그림 3 "I love you"와 "Do you love me"를 함께 넣을 때의 입력. 토큰마다 세 가지 벡터를 더한다

- "Here is the sentence I want embeddings for."를 토큰화하면 embeddings가 `em`, `##bed`, `##ding`, `##s`로 쪼개집니다. embeddings가 단어 집합에 없기 때문입니다(실습 2).
- 특별 토큰의 번호: [PAD] 0, [UNK] 100, [CLS] 101, [SEP] 102, [MASK] 103
- 두 문장을 길이 12로 넣으면 `token_type_ids`는 [0,0,0,0,0,1,1,1,1,1,0,0], `attention_mask`는 [1,1,1,1,1,1,1,1,1,1,0,0] 입니다.
- 입력 (1, 10) → 출력 (1, 10, 768). [CLS] 자리의 벡터는 (1, 768)입니다.
- BERT-Base의 파라미터는 109,482,240개입니다. 13장의 LSTM 분류기(100만 개 안팎)의 100배쯤입니다.

2) 사전 훈련: MLM과 NSP

정의

BERT는 위키백과(25억 단어)와 책(8억 단어)으로, 두 가지 문제를 동시에 풀며 학습했습니다.

(1) 마스크드 언어 모델(MLM). 입력 토큰의 15%를 골라 원래 단어를 맞히게 합니다. 고른 15%를 모두 [MASK]로 바꾸지는 않습니다.

고른 토큰 중	하는 일	예: The man went to the store
80%	[MASK]로 바꾼다	The man went to the [MASK]
10%	다른 단어로 바꾼다	The man went to the dog
10%	그대로 둔다	The man went to the store

왜 전부 [MASK]로 바꾸지 않을까? 실제로 쓸 때(파인 튜닝)는 입력에 [MASK]가 없습니다. 사전 훈련에서 [MASK]만 봤다면 "[MASK]가 있을 때만 그 자리를 신경 쓰는" 모델이 됩니다. 일부를 다른 단어로 바꾸거나 그대로 두면, 모델은 어느 자리가 문제인지 모르므로 모든 자리의 단어를 문맥에 비추어 살피게 됩니다.

손실은 고른 15%의 자리에서만 계산합니다. 나머지 85%의 자리는 출력이 나오지만 학습에 쓰지 않습니다.

(2) 다음 문장 예측(Next Sentence Prediction, NSP). 문장 두 개를 주고, 둘째 문장이 실제로 첫째 문장 바로 뒤에 오던 문장인지 맞히게 합니다. 절반은 실제로 이어지는 문장, 절반은 무작위로 고른 문장으로 만듭니다. [CLS] 자리의 출력 벡터로 둘 중 하나를 고릅니다.

왜 NSP도 학습할까? 질문에 답하기, 두 문장의 논리 관계 맞히기처럼 **두 글 사이의 관계**를 알아야 하는 문제가 많습니다. 단어를 맞히는 MLM만으로는 그것을 직접 배우지 않습니다.

예시 적용

"My dog is cute. He likes playing"에서 dog를 고르면 80%의 확률로 "My [MASK] is cute. he likes playing"이 입력이 되고, BERT는 [MASK] 자리의 출력 벡터로 dog를 맞혀야 합니다. NSP의 입력은 "[CLS] the man went to the store [SEP] he bought a gallon of milk [SEP]"(이어짐)와 "[CLS] the man went to the store [SEP] dogs are so cute [SEP]"(이어지지 않음) 같은 것입니다. 19-03과 19-04에서 실제로 실행합니다.

3) 파인 튜닝의 네 가지 유형

정의

사전 훈련된 BERT 위에 **출력층 하나**를 붙이고, 내 데이터로 BERT까지 함께 조금 더 학습합니다. 문제의 종류에 따라 BERT의 어느 출력을 쓰는지가 달라집니다.

유형	입력	쓰는 출력	예 (20장)
글 하나의 분류	[CLS] 글 [SEP]	[CLS] 벡터 → 선형층	뉴스의 긍정/부정 (20-01)
태깅	[CLS] 글 [SEP]	모든 토큰의 벡터 → 선형층	개체명 인식 (20-03)
글 두 개의 분류·회귀	[CLS] 글 A [SEP] 글 B [SEP]	[CLS] 벡터 → 선형층	자연어 추론 KorNLI (20-02)
질의 응답	[CLS] 질문 [SEP] 본문 [SEP]	모든 토큰의 벡터 → 답의 시작·끝 자리	기계 독해 (20-04)

13장의 다 대 일(분류)과 14장의 다 대 다(태깅)가 그대로 있습니다. RNN의 마지막 은닉 상태가 [CLS] 벡터로, 모든 시점의 은닉 상태가 모든 토큰의 벡터로 바뀌었을 뿐입니다.

예시 적용

영화 리뷰 "이 영화 정말 재밌어요"의 긍정/부정을 맞히려면, "[CLS] 이 영화 정말 재밌어요 [SEP]"를 넣어 나온 [CLS] 벡터 (768,)를 `nn.Linear(768, 2)` 에 넣으면 됩니다. 새로 학습할 것은 이 선형층의 가중치 1,538개와, 조금씩 고쳐지는 BERT의 가중치입니다.

비유

교양을 쌓은 신입 사원

신입 사원은 회사에 오기 전에 이미 수많은 글을 읽으며 말과 글을 익혔습니다. 그 과정에서 한 연습은 빈칸 채우기와 "이 문장 다음에 이 문장이 오는가"였습니다. 회사에서는 맡은 일에 맞는 양식 한 장(출력층)을 받아, 며칠 실습하면 일을 합니다. 같은 사람이 고객 문의 분류도, 계약서에서 이름 찾기도 할 수 있습니다.

입사 전의 독서 = = 사전 훈련
빈칸 채우기 연습 = = MLM
문장이 이어지는지 맞히기 = = NSP
일에 맞는 양식 한 장 = = 출력층
며칠의 실습 = = 파인 튜닝

실습 2

BERT의 토크나이저, 세 가지 입력, 임베딩, 출력의 모양

```python
import torch
from transformers import AutoModel, AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")
model.eval()

# 1) WordPiece 토크나이저: 단어 집합에 없는 단어는 아는 조각으로 쪼갠다
text = "Here is the sentence I want embeddings for."
print("토큰:", tokenizer.tokenize(text))
print("단어 집합의 크기:", tokenizer.vocab_size)
print("특별 토큰 번호:", {t: tokenizer.convert_tokens_to_ids(t)
                    for t in ["[PAD]", "[UNK]", "[CLS]", "[SEP]", "[MASK]"]})

# 2) 문장 하나를 넣을 때와 두 개를 넣을 때의 입력
one = tokenizer(text)
print("\n[문장 1개]", tokenizer.convert_ids_to_tokens(one["input_ids"]))
two = tokenizer("I love you", "Do you love me", padding="max_length", max_length=12)
print("[문장 2개]", tokenizer.convert_ids_to_tokens(two["input_ids"]))
print("  input_ids     :", two["input_ids"])
print("  token_type_ids:", two["token_type_ids"], "← 첫 문장 0, 둘째 문장 1")
print("  attention_mask:", two["attention_mask"], "← 패딩은 0")

# 3) 임베딩 층 세 개를 더한 것이 BERT의 입력이다
emb = model.embeddings
print("\n단어 임베딩  :", tuple(emb.word_embeddings.weight.shape))
print("위치 임베딩  :", tuple(emb.position_embeddings.weight.shape))
print("세그먼트 임베딩:", tuple(emb.token_type_embeddings.weight.shape))

# 4) 출력: 토큰마다 768칸짜리 벡터
inputs = tokenizer("I love you", "Do you love me", return_tensors="pt")
```

```python
with torch.no_grad():
    out = model(**inputs)
print("\n입력:", tuple(inputs["input_ids"].shape),
      "→ 출력:", tuple(out.last_hidden_state.shape))
print("[CLS] 자리의 벡터:", tuple(out.last_hidden_state[:, 0].shape),
      "← 분류에 쓰는 문장 전체의 요약")

# 5) 크기: 인코더 층 12개, 헤드 12개, 파라미터 약 1.1억 개
config = model.config
total = sum(p.numel() for p in model.parameters())
print(f"\n층 {config.num_hidden_layers}개, 은닉 크기 {config.hidden_size},",
      f"헤드 {config.num_attention_heads}개, 파라미터 {total:,}개")
```

실행 결과

```
토큰: ['here', 'is', 'the', 'sentence', 'i', 'want', 'em', '##bed', '##ding', '##s', 'for', '.']
단어 집합의 크기: 30522
특별 토큰 번호: {'[PAD]': 0, '[UNK]': 100, '[CLS]': 101, '[SEP]': 102, '[MASK]': 103}

[문장 1개] ['[CLS]', 'here', 'is', 'the', 'sentence', 'i', 'want', 'em', '##bed', '##ding', '##s', 'for',
'.', '[SEP]']
[문장 2개] ['[CLS]', 'i', 'love', 'you', '[SEP]', 'do', 'you', 'love', 'me', '[SEP]', '[PAD]', '[PAD]']
  input_ids     : [101, 1045, 2293, 2017, 102, 2079, 2017, 2293, 2033, 102, 0, 0]
  token_type_ids: [0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0] ← 첫 문장 0, 둘째 문장 1
  attention_mask: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0] ← 패딩은 0

단어 임베딩   : (30522, 768)
위치 임베딩   : (512, 768)
세그먼트 임베딩: (2, 768)

입력: (1, 10) → 출력: (1, 10, 768)
[CLS] 자리의 벡터: (1, 768) ← 분류에 쓰는 문장 전체의 요약

층 12개, 은닉 크기 768, 헤드 12개, 파라미터 109,482,240개
```

직접 해 보기 `tokenizer.tokenize("unbelievably")`와 `tokenizer.tokenize("서울")`을 실행해 보세요. 영어 모델에 한국어를 넣으면 어떻게 되나요?

확인 문제

19-02 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 BERT는 트랜스포머의 디코더를 쌓아 만든 모델이다. (O / X) 이름 _____

2 MLM에서 고른 15%의 토큰은 모두 [MASK]로 바뀐다. (O / X) 이름 _____

3 분류 문제에서 출력층에 넣는 BERT의 출력은? 이름 _____

① [CLS] 자리의 벡터 ② [SEP] 자리의 벡터 ③ 마지막 토큰의 벡터

4 문장 두 개를 구분하는 데 쓰는 임베딩은? 이름 _____

① 단어 임베딩 ② 위치 임베딩 ③ 세그먼트 임베딩

5 BERT-Base의 층 수는 _____, 벡터의 크기는 768이다. 이름 _____

6 실제 토큰은 1, 패딩은 0으로 표시해 주는 입력을 _____(이)라고 한다. 이름 _____

7 BERT의 사전 훈련과 파인 튜닝을 교양을 쌓은 신입 사원에 빗대어 설명해 보세요. 이름 _____

원문: 19-02 버트(BERT): GPT의 라이벌 — https://wikidocs.net/217013

19-03 BERT의 마스크드 언어 모델

핵심 한 줄

사전 훈련된 BERT에 [MASK]가 든 문장을 넣으면, [MASK] 자리의 출력 벡터가 선형층과 소프트맥스를 거쳐 단어 집합 전체의 확률이 되고, 그중 확률이 높은 단어가 빈칸의 답이다.

왜 배우나

19-02에서 말로 설명한 MLM을 실제 모델로 실행합니다. 허깅페이스의 transformers 라이브러리로 모델과 토크나이저를 내려받아 쓰는 방법도 여기서 처음 나오고, 20장과 21장 내내 같은 방식으로 씁니다.

1) 모델과 토크나이저 불러오기, 빈칸 맞히기

정의

- 허깅페이스(Hugging Face): 사전 훈련된 모델을 모아 둔 저장소이자, 그것을 불러 쓰는 `transformers` 라이브러리를 만든 곳.
- `AutoTokenizer.from_pretrained("모델 이름")` : 그 모델이 학습될 때 쓴 토크나이저를 불러옵니다. **모델과 토크나이저는 반드시 같은 이름으로 짝을 맞춥니다.** 단어 집합(토큰 ↔ 번호)이 모델마다 다르기 때문입니다.
- `AutoModelForMaskedLM.from_pretrained("모델 이름")` : BERT 본체 위에 "빈칸의 단어를 맞히는 출력층"이 얹힌 구조로 불러옵니다. 같은 BERT라도 풀 문제에 따라 `…ForMaskedLM` , `…ForNextSentencePrediction` , `…ForSequenceClassification` 처럼 얹는 출력층이 다릅니다.
- `pipeline("fill-mask", …)` : 토큰화 → 모델 → 확률 계산 → 상위 후보 고르기를 한 번에 해 주는 도구.

$$\text{tokens } (n,) \xrightarrow{\text{BERT}} (n,\ 768) \xrightarrow{\text{linear}} (n,\ 30522) \xrightarrow{\text{softmax at [MASK]}} \text{probabilities of 30522 words}$$

출력층은 모든 자리에 대해 단어 집합 크기만큼의 점수를 냅니다. 그중 [MASK]가 있는 자리의 점수만 꺼내 소프트맥스를 씌웁니다.

예시 적용



그림 4 "Soccer is a really fun [MASK]."에서 [MASK] 자리의 출력으로 단어를 맞힌다(실습 3의 실제 확률)

영어는 `bert-base-uncased` , 한국어는 `klue/bert-base` 로 실행했습니다(실습 3).

문장	상위 후보 (확률)
Soccer is a really fun [MASK].	sport 0.767, game 0.164, thing 0.016
The Avengers is a really fun [MASK].	game 0.311, story 0.121, movie 0.091
I went to [MASK] this morning.	work 0.304, bed 0.257, school 0.082
축구는 정말 재미있는 [MASK]다.	스포츠 0.896, 거 0.026, 경기 0.010
어벤져스는 정말 재미있는 [MASK]다.	영화 0.838, 거 0.028, 드라마 0.017
나는 오늘 아침에 [MASK]에 출근을 했다.	회사 0.080, [UNK] 0.061, 공장 0.017

- **문맥을 봅니다.** 뒤가 똑같은 "is a really fun [MASK]"인데 주어가 Soccer면 sport, Avengers면 game·story·movie가 나옵니다.

- **틀리는 경우.** 영어 모델은 The Avengers에 movie(0.091)보다 game(0.311)을 높게 줬습니다. 한국어 모델은 "어벤져스"에 영화를 0.838로 맞혔습니다. 교재는 더 큰 모델(bert-large-uncased)을 썼고, 이 자료는 내려받기 가벼운 base 모델을 써서 결과가 다를 수 있습니다.
- **자신 없을 때는 확률이 퍼집니다.** "[MASK]에 출근을 했다"는 회사가 1위지만 0.080뿐입니다. 올 수 있는 말(회사, 공장, 사무실, 지명)이 많기 때문입니다. 스포츠(0.896)처럼 몰린 경우와 대비됩니다.
- **파이프라인이 하는 일을 직접.** 입력 토큰은 9개, 출력 점수의 모양은 (1, 9, 30522), [MASK]는 6번 자리입니다. 그 자리의 점수에 소프트맥스를 씌워 상위 5개를 뽑으면 파이프라인의 결과와 같습니다.

비유

빈칸 채우기 시험

"축구는 정말 재미있는 ___다"라는 문제를 받으면, 빈칸의 앞과 뒤를 모두 읽고 들어갈 말의 후보를 떠올립니다. 확실하면 답이 하나로 좁혀지고, 애매하면 여러 답이 비슷하게 떠오릅니다. 이 시험을 책 수천만 권 분량으로 치른 것이 BERT입니다.

빈칸	= = [MASK]
앞뒤를 모두 읽기	= = 양방향 문맥 (셀프 어텐션)
떠오르는 후보와 확신의 정도	= = 단어마다의 확률

실습 3

영어와 한국어 BERT로 빈칸 맞히기

```
# (처음 한 번 한국어 모델 445MB 다운로드)
import torch
from transformers import AutoModelForMaskedLM, AutoTokenizer, pipeline

def top5(model_name, sentences):
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForMaskedLM.from_pretrained(model_name)
    fill = pipeline("fill-mask", model=model, tokenizer=tokenizer)
    for sentence in sentences:
        result = [(r["token_str"], round(r["score"], 3)) for r in fill(sentence)]
        print(f"  {sentence}\n    → {result}")
    return tokenizer, model

# 1) 영어: 교재는 bert-large-uncased(1.3GB)를 쓴다. 여기서는 bert-base-uncased
print("[영어 bert-base-uncased]")
tokenizer, model = top5("bert-base-uncased",
                        ["Soccer is a really fun [MASK].",
                         "The Avengers is a really fun [MASK].",
                         "I went to [MASK] this morning."])

# 2) 파이프라인이 하는 일을 직접: [MASK] 자리의 점수 → 소프트맥스 → 상위 5개
inputs = tokenizer("Soccer is a really fun [MASK].", return_tensors="pt")
print("\n입력 토큰:", tokenizer.convert_ids_to_tokens(inputs["input_ids"][0]))
with torch.no_grad():
    logits = model(**inputs).logits                # (1, 토큰 수, 단어 집합 크기)
position = inputs["input_ids"][0].tolist().index(tokenizer.mask_token_id)
probs = torch.softmax(logits[0, position], dim=0)    # 단어 30,522개의 확률
values, ids = probs.topk(5)
print("점수의 모양:", tuple(logits.shape), " [MASK]의 자리:", position)
print("직접 계산:", [(tokenizer.decode(i), round(v.item(), 3))
                  for v, i in zip(values, ids)])

# 3) 한국어: klue/bert-base
print("\n[한국어 klue/bert-base]")
top5("klue/bert-base", ["축구는 정말 재미있는 [MASK]다.",
                        "어벤져스는 정말 재미있는 [MASK]다.",
                        "나는 오늘 아침에 [MASK]에 출근을 했다."])
```

실행 결과

```
[영어 bert-base-uncased]
  Soccer is a really fun [MASK].
    → [('sport', 0.767), ('game', 0.164), ('thing', 0.016), ('day', 0.007), ('activity', 0.006)]
  The Avengers is a really fun [MASK].
    → [('game', 0.311), ('story', 0.121), ('movie', 0.091), ('adventure', 0.037), ('show', 0.033)]
  I went to [MASK] this morning.
    → [('work', 0.304), ('bed', 0.257), ('school', 0.082), ('sleep', 0.073), ('class', 0.062)]

입력 토큰: ['[CLS]', 'soccer', 'is', 'a', 'really', 'fun', '[MASK]', '.', '[SEP]']
점수의 모양: (1, 9, 30522)  [MASK]의 자리: 6
직접 계산: [('sport', 0.767), ('game', 0.164), ('thing', 0.016), ('day', 0.007), ('activity', 0.006)]

[한국어 klue/bert-base]
  축구는 정말 재미있는 [MASK]다.
    → [('스포츠', 0.896), ('거', 0.026), ('경기', 0.01), ('축구', 0.008), ('놀이', 0.008)]
```

```
어벤져스는 정말 재미있는 [MASK]다.
  → [('영화', 0.838), ('거', 0.028), ('드라마', 0.017), ('이야기', 0.015), ('장소', 0.009)]
나는 오늘 아침에 [MASK]에 출근을 했다.
  → [('회사', 0.08), ('[UNK]', 0.061), ('공장', 0.017), ('사무실', 0.016), ('서울', 0.015)]
```

직접 해 보기 "나는 [MASK]를 좋아한다."와 "나는 매운 [MASK]를 좋아한다."를 한국어 모델에 넣어 비교해 보세요. 단어 하나가 더해지면 후보가 어떻게 달라지나요?

확인 문제

19-03 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 모델과 토크나이저는 같은 모델 이름으로 불러와야 한다. (O / X) 이름 _____

2 BERT는 [MASK]의 앞에 있는 단어만 보고 빈칸을 맞힌다. (O / X) 이름 _____

3 빈칸의 단어를 정할 때 쓰는 것은? 이름 _____

① [CLS] 자리의 출력 ② [MASK] 자리의 출력 ③ 첫 단어의 출력

4 "축구는 정말 재미있는 [MASK]다."의 1위 후보는? 이름 _____

① 영화 ② 스포츠 ③ 회사

5 이 실습에서 쓴 한국어 BERT의 이름은 _____/bert-base다. 이름 _____

6 [MASK] 자리의 점수에 _____ 함수를 씌우면 단어마다의 확률이 된다. 이름 _____

7 확률이 한 단어에 몰린 경우와 여러 단어에 퍼진 경우를 실습 결과에서 하나씩 찾아 이유를 말해 보세요. 이름 _____

원문: 19-03 BERT의 마스크드 언어 모델 — https://wikidocs.net/217116 (교재의 bert-large-uncased(1.3GB)를 bert-base-uncased(440MB)로 바꿔 실행했습니다. 그래서 영어 결과의 순위가 교재와 다를 수 있습니다)

19-04 BERT를 이용한 다음 문장 예측

핵심 한 줄

문장 두 개를 [CLS] A [SEP] B [SEP]로 이어 넣으면, [CLS] 자리의 출력 벡터가 "B가 A 다음에 이어지는 문장인가"의 두 점수가 된다.

왜 배우나

BERT가 사전 훈련에서 푼 두 번째 문제입니다. 문장 두 개를 하나의 입력으로 만드는 방법과 `token_type_ids` 가 실제로 어떻게 생겼는지를 여기서 확인합니다. 20-02(두 문장의 관계 맞히기)와 20-04(질문과 본문)가 같은 입력 형식을 씁니다.

1) 두 문장을 하나의 입력으로

정의

- `tokenizer(문장 A, 문장 B)` 처럼 두 개를 넘기면 `[CLS] A [SEP] B [SEP]` 형태로 이어 줍니다.
- `token_type_ids` 는 [CLS]부터 첫 [SEP]까지 0, 그 뒤부터 마지막 [SEP]까지 1입니다.
- `BertForNextSentencePrediction` : BERT 본체 위에 [CLS] 벡터를 받아 점수 2개를 내는 출력층이 얹힌 구조. **0번이 "이어진다", 1번이 "이어지지 않는다"**입니다.

$$\texttt{[CLS]}\ A\ \texttt{[SEP]}\ B\ \texttt{[SEP]} \xrightarrow{\text{BERT}} \underbrace{h_{\texttt{[CLS]}}}_{(768,)} \xrightarrow{\text{linear}} (2,) \xrightarrow{\text{softmax}} [\,P(\text{next}),\ P(\text{not next})\,]$$

예시 적용

교재의 예문으로 실행했습니다(실습 4).

앞 문장	뒤 문장	이어질 확률
In Italy, pizza served in formal settings, such as at a restaurant, is presented unsliced.	pizza is eaten with the use of a knife and fork. …	1.000
	The sky is blue due to the shorter wavelength of blue light.	0.000
2002년 월드컵 축구대회는 일본과 공동으로 개최되었던 세계적인 큰 잔치입니다.	여행을 가보니 한국의 2002년 월드컵 축구대회의 준비는 완벽했습니다.	1.000
	극장가서 로맨스 영화를 보고싶어요	0.000

- 영어 예문은 토큰 38개가 되고, [SEP]는 23번과 37번 자리에 있습니다. `token_type_ids` 는 0이 24개([CLS]부터 첫 [SEP]까지), 1이 14개입니다.
- 네 경우 모두 확률이 1.000 아니면 0.000으로 아주 확실합니다. 피자 이야기 뒤에 하늘 이야기처럼 주제가 전혀 다르면 쉬운 문제이기 때문입니다.

이 문제가 너무 쉬워 학습에 큰 도움이 되지 않는다는 지적이 있어, BERT 뒤에 나온 RoBERTa 같은 모델은 NSP를 빼고 MLM만으로 사전 훈련합니다(이 내용은 교재에 없는 보충입니다).

비유

섞인 문장 카드 맞추기

글을 문장마다 카드로 잘라 섞은 뒤, 카드 두 장을 뽑아 "이 둘은 원래 붙어 있던 문장인가"를 맞힙니다. 같은 이야기를 하고 있고 뒤 문장이 앞 문장을 자연스럽게 받으면 "붙어 있었다"고 답합니다.

카드 두 장 = = 문장 A, 문장 B
두 장 사이의 구분선 = = [SEP], 세그먼트 임베딩
붙어 있었는지의 판단 = = [CLS] 벡터 → 점수 2개

실습 4 영어와 한국어 BERT로 다음 문장 예측하기

```python
import torch
from transformers import AutoTokenizer, BertForNextSentencePrediction

def nsp(model_name, first, candidates):
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = BertForNextSentencePrediction.from_pretrained(model_name)
    model.eval()
    print(f"[{model_name}] 앞 문장: {first}")
    for second in candidates:
        encoding = tokenizer(first, second, return_tensors="pt")
        with torch.no_grad():
            logits = model(**encoding).logits        # (1, 2): [이어짐, 아님]
        probs = torch.softmax(logits, dim=1)[0]
        label = "이어지는 문장" if probs[0] > probs[1] else "이어지지 않는 문장"
        print(f"  {second[:40]:40s} → {label} (이어질 확률 {probs[0]:.3f})")
    return tokenizer, encoding

# 1) 영어 (교재의 예문)
first = ("In Italy, pizza served in formal settings, such as at a restaurant, "
         "is presented unsliced.")
tokenizer, encoding = nsp("bert-base-uncased", first, [
    "pizza is eaten with the use of a knife and fork. In casual settings, however, "
    "it is cut into wedges to be eaten while held in the hand.",
    "The sky is blue due to the shorter wavelength of blue light."])

# 2) 두 문장이 어떻게 하나의 입력이 되는가
tokens = tokenizer.convert_ids_to_tokens(encoding["input_ids"][0])
types = encoding["token_type_ids"][0].tolist()
print("\n토큰 수:", len(tokens))
print("앞부분:", tokens[:6], "...")
print("[SEP]의 자리:", [i for i, t in enumerate(tokens) if t == "[SEP]"])
print("token_type_ids: 0이", types.count(0), "개, 1이", types.count(1), "개")

# 3) 한국어 (교재의 예문)
print()
first = "2002년 월드컵 축구대회는 일본과 공동으로 개최되었던 세계적인 큰 잔치입니다."
nsp("klue/bert-base", first, [
    "여행을 가보니 한국의 2002년 월드컵 축구대회의 준비는 완벽했습니다.",
    "극장가서 로맨스 영화를 보고싶어요"])
```

실행 결과

```
[bert-base-uncased] 앞 문장: In Italy, pizza served in formal settings, such as at a restaurant, is presented
unsliced.
  pizza is eaten with the use of a knife a → 이어지는 문장 (이어질 확률 1.000)
  The sky is blue due to the shorter wavel → 이어지지 않는 문장 (이어질 확률 0.000)

토큰 수: 38
앞부분: ['[CLS]', 'in', 'italy', ',', 'pizza', 'served'] ...
[SEP]의 자리: [23, 37]
token_type_ids: 0이 24 개, 1이 14 개

[klue/bert-base] 앞 문장: 2002년 월드컵 축구대회는 일본과 공동으로 개최되었던 세계적인 큰 잔치입니다.
  여행을 가보니 한국의 2002년 월드컵 축구대회의 준비는 완벽했습니다.  → 이어지는 문장 (이어질 확률 1.000)
  극장가서 로맨스 영화를 보고싶어요                          → 이어지지 않는 문장 (이어질 확률 0.000)
```

직접 해 보기 주제는 같지만 이어지지는 않는 문장을 넣어 보세요. 예: 앞 문장이 월드컵 이야기일 때 "축구공은 둥글다." 모델은 어떻게 답하나요?

19-04 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 다음 문장 예측에서는 [CLS] 자리의 출력 벡터를 쓴다. (O / X) 이름 _____

2 두 문장을 넣을 때 [SEP]는 한 번만 들어간다. (O / X) 이름 _____

3 둘째 문장의 토큰에 붙는 `token_type_ids` 의 값은? 이름 _____

① 0 ② 1 ③ 102

4 다음 문장 예측의 출력은 몇 개의 점수인가요? 이름 _____

① 1개 ② 2개 ③ 30,522개

5 다음 문장 예측을 영어 약자로 _____라고 한다. 이름 _____

6 두 문장의 입력 형식은 [CLS] A _____ B [SEP]다. 이름 _____

7 BERT가 MLM뿐 아니라 NSP도 학습한 이유를 설명해 보세요. 이름 _____

원문: 19-04 BERT를 이용한 다음 문장 예측 — https://wikidocs.net/217128

19-05 문장과 문서 임베딩을 위한 SBERT

핵심 한 줄

SBERT는 문장 벡터끼리의 코사인 유사도가 뜻의 비슷한 정도가 되도록 BERT를 파인 튜닝한 모델이고, 문장 검색과 추천에 쓴다.

왜 배우나

11장에서는 TF-IDF 벡터로 비슷한 문서를 찾았습니다. 단어가 겹쳐야만 비슷하다고 보는 방법이었습니다. 뜻이 같은데 단어가 다른 문장을 찾으려면 문장의 **뜻**을 담은 벡터가 필요합니다. BERT가 그런 벡터를 줄 것 같지만 그대로 쓰면 잘 되지 않습니다. 그 이유와 해결책이 SBERT이고, 20-06의 챗봇과 20-07의 검색기, 24장의 RAG가 이 위에 만들어집니다.

1) BERT에서 문장 벡터를 얻는 방법

정의

BERT의 출력은 토큰마다의 벡터 (n, 768)입니다. 문장 하나를 벡터 하나로 만드는 방법은 세 가지입니다.

- **[CLS] 벡터**: [CLS] 자리의 출력 벡터를 문장 벡터로 본다.
- **평균 풀링(mean pooling)**: 모든 토큰의 출력 벡터를 평균 낸다. 모든 단어의 뜻을 고르게 반영한다. 패딩 자리는 평균에서 뺀다.
- **맥스 풀링(max pooling)**: 칸마다 가장 큰 값을 고른다. 두드러진 단어의 뜻을 반영한다(13-04).

그대로 쓰면 왜 잘 안 될까? BERT는 빈칸 맞히기와 다음 문장 맞히기로 학습했습니다. "문장 벡터끼리의 코사인 유사도가 뜻의 비슷한 정도가 되어라"라고 학습한 적은 없습니다. 그래서 사전 훈련만 된 BERT의 문장 벡터는 어떤 두 문장을 넣어도 유사도가 대체로 높게 나와, 비슷한 문장과 다른 문장을 잘 가르지 못합니다.

예시 적용

"I love you"를 BERT에 넣으면 [CLS], i, love, you, [SEP] 다섯 토큰의 벡터 (5, 768)이 나옵니다. [CLS] 벡터를 쓰면 첫 행 하나를, 평균 풀링을 쓰면 다섯 행의 평균을 문장 벡터 (768,)로 삼습니다.

2) SBERT의 학습 방법

정의

- SBERT(Sentence-BERT): 문장 벡터가 뜻을 잘 담도록 BERT를 문장 쌍 데이터로 파인 튜닝한 모델.
- 문장 A와 B를 **같은 BERT에 따로따로** 넣고, 각각 풀링해 문장 벡터 u, v를 얻습니다.

	문장 쌍 분류로 학습 (NLI)	문장 쌍 회귀로 학습 (STS)
데이터	두 문장 + 관계 (함의 / 모순 / 중립)	두 문장 + 유사도 점수 (0~5)
계산	$h = (u;\ v;\ \lvert u - v \rvert)$를 이어 붙여 선형층 → 소프트맥스	u와 v의 코사인 유사도
손실	크로스 엔트로피	레이블 유사도와의 평균 제곱 오차(MSE)

- NLI(Natural Language Inference, 자연어 추론): 두 문장이 함의(A가 참이면 B도 참), 모순, 중립 가운데 어떤 관계인지 맞히는 문제.
- STS(Semantic Textual Similarity): 두 문장의 뜻이 얼마나 비슷한지 점수로 맞히는 문제.
- $\lvert u - v \rvert$는 두 벡터의 칸마다의 차이의 절댓값입니다. u, v의 크기가 n이면 h의 크기는 $3n$입니다.
- 둘 중 하나로만 학습할 수도 있고, NLI로 먼저 학습한 뒤 STS로 이어서 학습할 수도 있습니다.

왜 이렇게 하면 될까? 손실을 줄이려면, 뜻이 같은 문장 쌍은 u와 v가 가까워지고 모순인 쌍은 멀어져야 합니다. 그렇게 BERT의 가중치가 고쳐지면서 "가까운 벡터 = 비슷한 뜻"이 됩니다.

왜 문장을 따로 넣을까? BERT에 두 문장을 함께 넣어도([CLS] A [SEP] B [SEP]) 유사도를 구할 수 있고 더 정확합니다. 하지만 문장 1만 개 가운데 가장 비슷한 쌍을 찾으려면 약 5,000만 쌍을 하나씩 BERT에 넣어야 합니다. 문장마다 벡터를 한 번씩만 구해 두면(1만 번), 나머지는 벡터끼리의 코사인 유사도 계산이라 훨씬 빠릅니다.

예시 적용

문장 넷을 사전 훈련만 된 BERT(`klue/bert-base`)와 한국어 SBERT(`jhgan/ko-sroberta-multitask`)에 넣어 비교했습니다(실습 5).

- 날씨: "오늘 날씨가 정말 좋네요" / 화창: "하늘이 맑고 화창한 하루입니다" → 뜻이 비슷하고 겹치는 단어가 없다
- 주식: "오늘 주식 시장이 크게 올랐습니다" / 급등: "주가가 급등했어요" → 뜻이 비슷하다
- 날씨와 주식은 "오늘"이 겹치지만 뜻이 다르다

코사인 유사도	날씨-화창 (비슷)	주식-급등 (비슷)	날씨-주식 (다름)	화창-급등 (다름)
BERT [CLS]	0.844	0.800	0.687	0.592
BERT 평균 풀링	0.755	0.737	0.695	0.499
SBERT	0.745	0.725	0.301	0.238

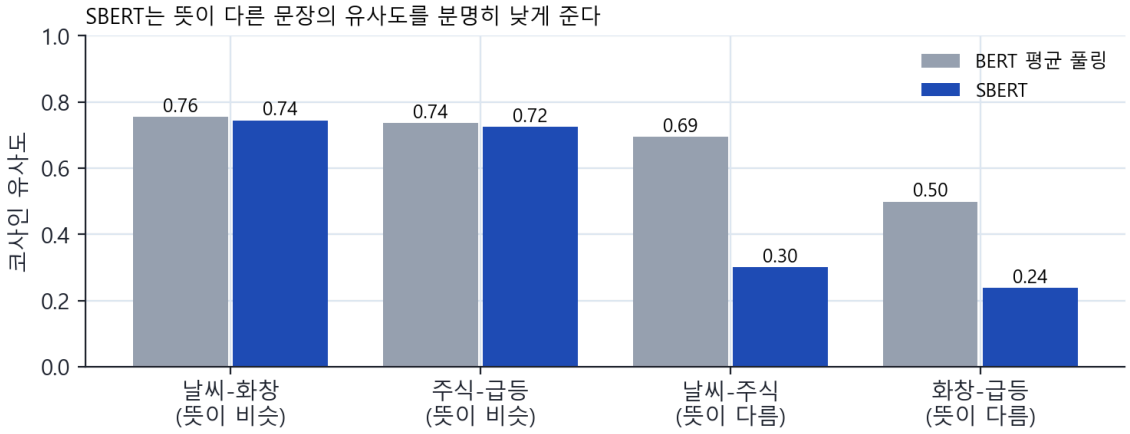


그림 5 뜻이 비슷한 쌍의 유사도는 둘이 비슷하지만, 뜻이 다른 쌍에서 SBERT가 훨씬 낮은 값을 준다

- **BERT는 모두 높습니다.** 평균 풀링에서 뜻이 다른 날씨-주식(0.695)이 뜻이 비슷한 주식-급등(0.737)과 거의 차이가 없습니다. 이 숫자로는 비슷한 문장을 가려낼 수 없습니다.
- **SBERT는 가릅니다.** 비슷한 쌍은 0.73~0.75, 다른 쌍은 0.24~0.30으로 뚜렷이 나뉩니다.
- **검색.** "비가 올 것 같아요"와 가장 비슷한 문장으로 날씨 문장 둘(0.307, 0.219)이 주식 문장 둘(0.077, 0.055)보다 위에 나옵니다. 겹치는 단어는 하나도 없습니다. 다만 "비가 온다"와 "날씨가 좋다"는 뜻이 반대이기도 해서 유사도 자체는 낮습니다.

비유

책마다 한 줄 요약 카드를 만들어 두는 사서

비슷한 책을 찾을 때마다 두 권을 나란히 놓고 처음부터 끝까지 비교하면 정확하지만, 책이 만 권이면 끝이 없습니다. 책마다 한 줄 요약 카드를 미리 만들어 두면, 카드끼리만 비교하면 됩니다. 단, 카드는 "비슷한 책은 비슷한 요약이 되도록" 쓰는 연습을 한 사람이 써야 쓸모가 있습니다.

한 줄 요약 카드 = = 문장 벡터
카드끼리 비교 = = 코사인 유사도
요약 쓰는 연습 = = NLI, STS로 파인 튜닝
두 권을 통째로 비교 = = 두 문장을 BERT에 함께 넣기

실습 5

BERT와 SBERT의 문장 벡터로 유사도 비교하기

```
# 코랩에서는 먼저:  !pip install sentence-transformers   (모델 440MB 다운로드)
import torch
from sentence_transformers import SentenceTransformer
from transformers import AutoModel, AutoTokenizer

sentences = ["오늘 날씨가 정말 좋네요",              # 0
             "하늘이 맑고 화창한 하루입니다",          # 1  (0과 뜻이 비슷)
             "오늘 주식 시장이 크게 올랐습니다",        # 2  (0과 '오늘'만 겹침)
             "주가가 급등했어요"]                    # 3  (2와 뜻이 비슷)

# 1) 사전 훈련만 된 BERT에서 문장 벡터 얻기: [CLS] 벡터, 평균 풀링
tokenizer = AutoTokenizer.from_pretrained("klue/bert-base")
bert = AutoModel.from_pretrained("klue/bert-base")
inputs = tokenizer(sentences, padding=True, return_tensors="pt")
with torch.no_grad():
```

```
    hidden = bert(**inputs).last_hidden_state        # (4, 토큰 수, 768)
cls = hidden[:, 0]                                   # [CLS] 자리의 벡터
mask = inputs["attention_mask"].unsqueeze(-1)        # 패딩은 평균에서 뺀다
mean = (hidden * mask).sum(dim=1) / mask.sum(dim=1)
print("BERT 출력:", tuple(hidden.shape), "→ [CLS]:", tuple(cls.shape),
      " 평균 풀링:", tuple(mean.shape))

# 2) SBERT: 문장 벡터끼리의 유사도가 뜻을 반영하도록 파인 튜닝한 모델
sbert = SentenceTransformer("jhgan/ko-sroberta-multitask")
emb = torch.tensor(sbert.encode(sentences))
print("SBERT 문장 벡터:", tuple(emb.shape))

def show(name, vectors):
    a, b = vectors.unsqueeze(1), vectors.unsqueeze(0)
    sim = torch.cosine_similarity(a, b, dim=-1)       # 모든 쌍의 유사도 (4, 4)
    print(f"{name:14s} 날씨-화창 {sim[0, 1]:.3f} | 날씨-주식 {sim[0, 2]:.3f} |",
          f"주식-급등 {sim[2, 3]:.3f} | 화창-급등 {sim[1, 3]:.3f}")

print("\n문장 쌍의 코사인 유사도 (뜻이 비슷한 쌍: 날씨-화창, 주식-급등)")
show("BERT [CLS]", cls)
show("BERT 평균 풀링", mean)
show("SBERT", emb)

# 3) SBERT로 가장 비슷한 문장 찾기
query = "비가 올 것 같아요"
q = torch.tensor(sbert.encode([query]))
scores = torch.cosine_similarity(q, emb)
print(f"\n'{query}'와 가장 비슷한 문장:")
for i in scores.argsort(descending=True):
    print(f"  {scores[i]:.3f}  {sentences[i]}")
```

실행 결과

```
BERT 출력: (4, 11, 768) → [CLS]: (4, 768)  평균 풀링: (4, 768)
SBERT 문장 벡터: (4, 768)

문장 쌍의 코사인 유사도 (뜻이 비슷한 쌍: 날씨-화창, 주식-급등)
BERT [CLS]     날씨-화창 0.844 | 날씨-주식 0.687 | 주식-급등 0.800 | 화창-급등 0.592
BERT 평균 풀링      날씨-화창 0.755 | 날씨-주식 0.695 | 주식-급등 0.737 | 화창-급등 0.499
SBERT          날씨-화창 0.745 | 날씨-주식 0.301 | 주식-급등 0.725 | 화창-급등 0.238

'비가 올 것 같아요'와 가장 비슷한 문장:
  0.307  오늘 날씨가 정말 좋네요
  0.219  하늘이 맑고 화창한 하루입니다
  0.077  주가가 급등했어요
  0.055  오늘 주식 시장이 크게 올랐습니다
```

직접 해 보기 `sentences`에 직접 쓴 문장을 더해 보세요. 뜻은 같고 말투만 다른 두 문장("밥 먹었어?", "식사하셨습니까?")의 SBERT 유사도는 얼마인가요?

확인 문제

19-05 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 평균 풀링은 모든 토큰의 출력 벡터를 평균 내어 문장 벡터를 만든다. (O / X) 이름 _____

2 사전 훈련만 된 BERT의 문장 벡터는 뜻이 다른 문장끼리도 유사도가 높게 나오는 편이다. (O / X) 이름 _____

3 SBERT를 학습할 때 문장 A와 B를 넣는 방법은? 이름 _____

① 같은 BERT에 따로따로 ② 한 입력으로 이어서 ③ 서로 다른 두 모델에

4 STS 데이터로 학습할 때 문장 벡터 u, v로 계산하는 것은? 이름 _____

① 코사인 유사도 ② 두 벡터의 합 ③ 다음 단어의 확률

5 두 문장이 함의, 모순, 중립 중 어떤 관계인지 맞히는 문제를 영어 약자로 _____라고 한다. 이름 _____

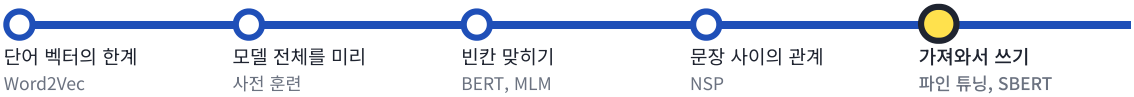
6 NLI로 학습할 때 출력층에 넣는 벡터는 $(u;\ v;\ _____)$를 이어 붙인 것이다. 이름 _____

7 문장 벡터를 미리 구해 두는 방식이 왜 빠른지, 요약 카드를 만드는 사서에 빗대어 설명해 보세요. 이름 _____

원문: 19-05 문장과 문서 임베딩을 위한 SBERT — https://wikidocs.net/217016 (교재에는 실습이 없어, 20-06에서 쓰는 한국어 SBERT 모델로 비교 실습을 추가했습니다. "왜 문장을 따로 넣는가"의 설명은 교재에 없는 보충입니다)

오늘의 요약

다시 보기



정의 모음

개념	정의	예시
사전 훈련 / 파인 튜닝	큰 글로 미리 학습 / 내 데이터로 조금 더 학습	33억 단어 → 리뷰 수만 개
BERT	트랜스포머 인코더 12층, 양방향	(n,) → (n, 768)
입력	단어 + 위치 + 세그먼트 임베딩	[CLS] A [SEP] B [SEP]
MLM	가린 단어를 앞뒤 문맥으로 맞히기	fun [MASK] → sport 0.767
NSP	둘째 문장이 이어지는 문장인지 맞히기	피자 → 하늘: 0.000
파인 튜닝의 유형	분류, 태깅, 글 쌍의 분류, 질의 응답	[CLS] 벡터 → 선형층
SBERT	문장 벡터의 유사도가 뜻을 반영하도록 학습	날씨-주식: 0.695 → 0.301

다음

다음 자료: 20장 BERT 학습하기 (10/16 자료 2/5)

오늘은 이미 학습된 BERT를 그대로 썼습니다. 다음 자료에서는 BERT 위에 출력층을 붙여 내 데이터로 파인 튜닝합니다. 뉴스의 긍정/부정 분류, 두 문장의 관계 맞히기, 개체명 인식, 질문에 답하기를 차례로 풉니다.

정답과 해설

19-01 자연어 처리에서의 사전 훈련

1. X — 한 단어에 벡터 하나입니다.
2. O — 글 자체에서 문제와 정답을 만듭니다.
3. ②
4. ①
5. 파인 튜닝 (미세 조정)
6. 0.300
7. 예시 답: 단어장만 외우면 단어마다 뜻 하나만 알지만(워드 임베딩), 책을 많이 읽으면 문장 속에서 뜻을 구분하고 글을 읽는 법 자체를 익힌다(언어 모델의 사전 훈련). 새 일을 배울 때는 그 일의 말만 조금 익히면 된다(파인 튜닝).

19-02 버트(BERT)

1. X — 인코더를 쌓았습니다. 디코더를 쌓은 것은 GPT입니다.
2. X — 80%만 [MASK], 10%는 다른 단어, 10%는 그대로입니다.
3. ①
4. ③
5. 12
6. attention_mask (어텐션 마스크)
7. 예시 답: 입사 전에 수많은 글을 읽으며 빈칸 채우기와 문장 이어지는지 맞히기로 말을 익힌 것이 사전 훈련이고, 회사에서 일에 맞는 양식(출력층)을 받아 며칠 실습하는 것이 파인 튜닝이다.

19-03 BERT의 마스크드 언어 모델

1. O — 단어 집합이 모델마다 다릅니다.
2. X — 앞뒤를 모두 봅니다.
3. ②
4. ② — 0.896
5. klue
6. 소프트맥스
7. 예시 답: "축구는 정말 재미있는 [MASK]다"는 스포츠에 0.896이 몰렸다. 들어갈 말이 거의 정해져 있기 때문이다. "[MASK]에 출근을 했다"는 1위인 회사가 0.080뿐이다. 회사, 공장, 사무실, 지명 등 올 수 있는 말이 많기 때문이다.

19-04 BERT를 이용한 다음 문장 예측

1. O
2. X — 두 문장의 끝에 하나씩, 두 번 들어갑니다.
3. ②
4. ② — 이어진다 / 이어지지 않는다
5. NSP
6. [SEP]
7. 예시 답: 질문에 답하기나 두 문장의 논리 관계 맞히기처럼 두 글 사이의 관계를 알아야 하는 문제가 많은데, 단어를 맞히는 MLM만으로는 그것을 직접 배우지 않기 때문이다.

19-05 문장과 문서 임베딩을 위한 SBERT

1. O
2. O — 실습에서 뜻이 다른 쌍도 0.5~0.7이었습니다.
3. ①
4. ①
5. NLI
6. |u - v|
7. 예시 답: 책 두 권을 통째로 비교하면 정확하지만 책이 많으면 끝이 없다. 책마다 요약 카드를 한 번씩만 만들어 두면 카드끼리만 비교하면 되듯, 문장마다 벡터를 한 번만 구해 두고 벡터끼리의 코사인 유사도만 계산하면 된다.

용어 정리

용어	정의
사전 훈련 / 파인 튜닝	큰 데이터로 모델을 미리 학습시키는 것 / 그 모델을 내 데이터로 조금 더 학습시키는 것
BERT	트랜스포머 인코더를 쌓고 MLM과 NSP로 사전 훈련한 양방향 언어 모델
[CLS] / [SEP] / [MASK]	입력의 시작이자 분류용 / 문장의 끝 / 가린 단어의 자리
WordPiece	BERT가 쓰는 서브워드 토크나이저. 단어 중간의 조각에 ##을 붙인다
위치 임베딩 / 세그먼트 임베딩	몇 번째 토큰인지 / 몇째 문장인지를 나타내는 학습되는 벡터
input_ids / token_type_ids / attention_mask	토큰의 번호 / 문장 구분(0, 1) / 실제 토큰(1)과 패딩(0)의 구분
마스크드 언어 모델 (MLM)	가린 단어를 앞뒤 문맥으로 맞히는 학습 방법
다음 문장 예측 (NSP)	둘째 문장이 첫째 문장에 이어지는 문장인지 맞히는 학습 방법
허깅페이스 transformers	사전 훈련된 모델을 불러 쓰는 라이브러리
평균 풀링	토큰 벡터들을 평균 내어 문장 벡터 하나를 만드는 것
SBERT	문장 벡터의 유사도가 뜻을 반영하도록 BERT를 파인 튜닝한 모델
NLI / STS	두 문장의 논리 관계를 맞히는 문제 / 두 문장의 유사도 점수를 맞히는 문제